CESDE

Santiago Yosa

Ingeniero Sistemas de Información

CESDE | comfama

Unidad II

HERENCIA POO

¿Qué es herencia?

Clase base y clase derivada uso de EXTENDS

Concepto: Diferencia entre herencia simple y múltiple

SISTEMA CONTROL DE VERSIONES

-Utilizar git/GitHub para versionar código realizado

Fundamentos de POO

Situación disparadora o ejemplo del mundo real:

Imagina que te contratan para desarrollar el sistema de un Hospital. Te piden crear una clase Medico con los atributos: nombre, documento, edad y especialidad. Lo haces perfecto.

Al día siguiente, te piden crear una clase Paciente. Sus atributos son: nombre, documento, edad y numeroHistorial. Copias y pegas el código del médico y le cambias el último atributo.

Luego te piden crear un Enfermero, un Administrativo, un Cirujano... Terminas con 20 clases copiadas y pegadas.

Un día, el gobierno cambia la ley y el documento ahora debe llamarse idNacional. Tienes que abrir 20 archivos y modificar todo a mano. ¡Es un infierno de mantenimiento!

La Solución (Herencia): ¿Qué tienen en común todas estas personas? Que todas son una Persona. La Herencia nos permite crear una **Clase Base** (Persona) con los datos comunes (nombre, documento, edad), y luego crear **Clases Derivadas** (Medico, Paciente) que automáticamente *heredan* todo eso, y solo le añadimos lo que las hace especiales. Si la ley cambia, solo modificas la clase Persona y todo el sistema se actualiza mágicamente.

¿Qué es la Herencia?

Es un mecanismo que permite que una clase (llamada **clase derivada** o **subclase**) herede las propiedades (atributos) y los comportamientos (métodos) de otra clase (la **clase base** o **superclase**).

La herencia modela la relación **"es-un"**. Por ejemplo, un Coche "es-un" Vehiculo, y una Motocicleta "es-un" Vehiculo. Esto significa que una subclase es una versión más especializada o concreta de su superclase.

Piensa en la biología.

Los **mamíferos** son una clase. Los **perros** y los **gatos** son subclases de mamíferos. Todos los perros y gatos tienen las características de un mamífero (sangre caliente, espina dorsal, glándulas mamarias), pero además tienen sus propias características únicas (ladrar vs. maullar). La herencia en POO funciona exactamente igual.

Clase Base y Clase Derivada: Uso de extends

Clase Base (Superclase o Padre): La clase general que contiene el código común.
Clase Derivada (Subclase o Hijo): La clase especializada que hereda de la Base.
Palabra clave extends: Es el conector mágico en Java que dice "Esta clase extiende (hereda de) esta otra".

- **Clase Base (Vehiculo):** Define los atributos y métodos comunes a todos los tipos de vehículos.

- **Clase Derivada (Coche):** Extiende la clase Vehiculo. Hereda la funcionalidad común y puede añadir sus propios atributos (numeroDePuertas) o métodos.

extends en acción: Al usar extends, la clase Coche obtiene una copia de todo el código público y protegido de la clase Vehiculo sin tener que reescribirlo. Esto promueve la **reutilización de código** y la **cohesión**.

```java
// Clase Base o Superclase
public class Vehiculo {
    String marca;
    int year;

    public void acelerar() {
        System.out.println("El vehículo está acelerando.");
    }
}
```

```java
// Clase Derivada o Subclase
public class Coche extends Vehiculo {
    int numeroDePuertas;
    // La clase Coche hereda automáticamente 'marca', 'year' y el método 'acelerar()'.
}
```

Concepto: Diferencia entre Herencia Simple y Múltiple

Herencia Simple: Una clase puede heredar **solo de una única superclase**. Esto crea una jerarquía clara y lineal. Java solo soporta herencia simple de clases.

Ejemplo: Coche -> Vehiculo Motocicleta -> Vehiculo

Herencia Múltiple: Una clase hereda de **dos o más superclases**. Esto puede llevar a problemas de diseño, donde la ambigüedad en los métodos heredados causa confusión.

Java **no permite** la herencia múltiple de clases. En su lugar, usa **interfaces** para lograr un comportamiento similar de manera más segura y flexible.

Ejemplo: Un Hidroavion hereda de Barco y de Avion. **Java prohíbe esto en clases** para evitar el "Problema del Diamante" (si Barco y Avion tienen un método moverse(), ¿cuál usaría el Hidroavion?).

Si la herencia es tan útil, ¿por qué Java no permite la herencia múltiple?

La herencia múltiple puede causar ambigüedad y complejidad, especialmente en el "problema del diamante". Imagina que las clases A y B tienen un método ejecutar(), y la clase C hereda de ambas. Cuando llamas a C.ejecutar(), ¿qué método debe usar? Para evitar esta ambigüedad, Java optó por la simplicidad de la herencia simple y utiliza las interfaces para permitir que las clases implementen múltiples "comportamientos" sin los problemas de la herencia de estado.

¿Qué se hereda y qué no se hereda?

Una clase derivada hereda todos los **miembros no privados** de su superclase (atributos y métodos public y protected). **Los miembros privados no se heredan**, ya que su propósito es ser accesibles únicamente dentro de su propia clase. Sin embargo, las subclases pueden acceder a ellos a través de los métodos públicos o protegidos de la superclase (por ejemplo, getNombre() del padre).

La palabra clave final

Si no desea que otras clases hereden de una clase, utilice la final palabra clave:

```
final class Vehicle {
  ...
}

class Car extends Vehicle {
  ...
}
```

Demostrativo

Vamos a codificar el ecosistema de una empresa tecnológica.

```
public class Trabajador {
    private String nombre;
    private double salarioBase;

    public Trabajador(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    public void cobrar() {
        System.out.println(nombre + " ha cobrado su salario de $" + salarioBase);
    }

    // Getters para que los hijos puedan ver datos privados
    public String getNombre() { return nombre; }
    public double getSalarioBase() { return salarioBase; }
}
```

La Subclase (La especialización)

```
public class Programador extends Trabajador {
   private String lenguajeDominante;

   public Programador(String nombre, double salarioBase, String lenguaje) {
      super(nombre, salarioBase); // 1. Alimentamos al Padre
      this.lenguajeDominante = lenguaje; // 2. Alimentamos al Hijo
   }

   public void escribirCodigo() {
      // Usamos getNombre() porque 'nombre' es private en el Padre
      System.out.println(getNombre() + " está programando en " + lenguajeDominante);
   }
}
```

El Main (La ejecución)

```
public class MainEmpresa {
   public static void main(String[] args) {
      Programador pro = new Programador("Ana", 3000.0, "Java");

      pro.cobrar(); // ¡Magia! Este método no está en Programador, viene del Padre.
      pro.escribirCodigo(); // Método propio del hijo.
   }
}
```

Práctica Acompañada - Learning by Doing - El Concesionario

Vamos a modelar vehículos.

1. Crea una clase base Vehiculo con marca (String) y un método encender().
2. Crea una clase Moto que herede de Vehiculo. Añade el atributo cilindraje (int) y un método hacerCaballito().
3. Pruébalo en tu Main.

Solución de Errores Comunes

1. **Error de Sintaxis:** Implicit super constructor is undefined.
   - ***Causa:*** En la clase hija no pusiste super() y el padre no tiene un constructor vacío.
   - ***Solución:*** Asegúrate de que la primera línea del constructor hijo sea super(argumentosDelPadre);

2. **Error de Acceso:** The field Persona.nombre is not visible.
   - ***Causa:*** Intentaste hacer this.nombre en el hijo, pero en el padre es private.
   - ***Solución:*** En el padre, cámbialo a protected, o en el hijo usa this.getNombre().

3. **Mala aplicación de Herencia:** *Causa:* Hacer que Llantas herede de Auto.
   - ***Solución:*** Recordar la regla "ES UN". Una llanta no "es un" auto. Eso es composición (el auto tiene llantas).

Integración con Git y GitHub

Nuestro código vive en la máquina, pero un desarrollador profesional guarda su trabajo en un repositorio remoto (GitHub). Vamos a respaldar tu código de herencia.

Crear el repositorio en GitHub: (Desde el navegador, clic en "New Repository", sin README).

Vamos a crear el repositorio, pero antes vamos a agregar nuestro .gitignore.

En la terminal dentro de tu proyecto): Puede ser en el IDE o en la carpeta.

```
git init
git checkout -b main
git add .
git commit -m "feat: implementar jerarquia de herencia en Vehiculos y Dispositivos"
git remote add origin # Pegar el link que da GitHub:
git push -u origin main
```

Ejercicio Independiente Aplicado

Instrucciones:

1. Diseña una clase base llamada MaterialBibliografico con: titulo, autor y un método mostrarDetalles().
2. Diseña una clase derivada Libro que añada el atributo numeroPaginas.
3. Diseña una clase derivada Revista que añada el atributo numeroEdicion.
4. En el Main, crea un Libro y una Revista y llama al método mostrarDetalles() en ambos.
5. **Requisito de entrega:** Debes subir este proyecto a un nuevo repositorio público en GitHub y entregar el enlace (URL). El último commit debe llamarse "feat: culminacion de sistema de biblioteca".

Criterios de Logro:

- Uso correcto de extends y super().
- Uso correcto del encapsulamiento (private en atributos, getters/setters si es necesario).
- Evidencia de un repositorio limpio en GitHub (sin archivos del IDE gracias al .gitignore).

¿Debería saber algo más?

Concepto	Nivel	Pregunta que responde	Ejemplo Java
Paradigma	Mentalidad	¿Cómo percibo el problema?	Orientado a Objetos (POO)
Arquitectura	Estructura	¿Cómo organizo las partes?	Capas (Controller-Service-Repository)
Patrón	Solución Táctica	¿Cómo resuelvo este problema específico?	Singleton, DTO, Repository
Convención	Estilo	¿Cómo escribo para que me entiendan?	camelCase, nombres en inglés

CESDE | comfama